

OfficeEditor Phase 4: Replacing Legacy Rendering Pipeline

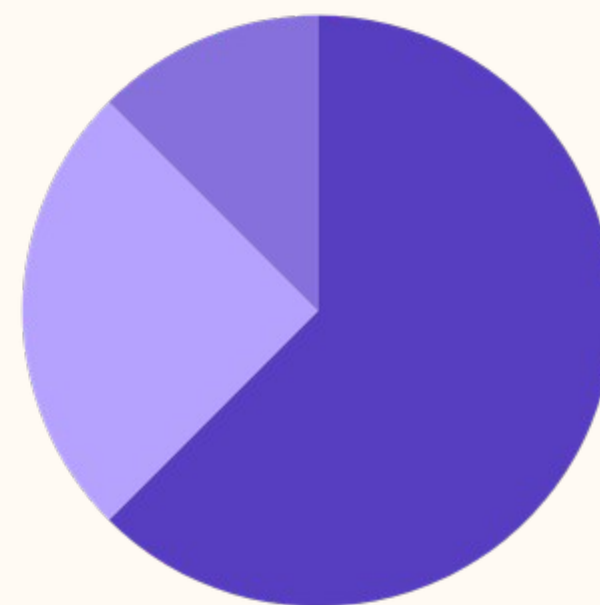
Internal Presentation

July 2026

Phase 4 Pitch Deck

Presented By:
Engineering Team

Presented Date:
July 18, 2026



Project Overview & Core Thesis

What is OfficeEditor?

OfficeEditor replaces the legacy slide-rendering microservice with an in-process C# library, eliminating process management overhead and LibreOffice dependencies entirely.

Core Thesis: Instant preview beats perfect preview. A fast, iterative preview at ~12% RMSE is far more valuable to users than a slow, near-perfect render at 2% RMSE that takes 15 seconds.

Preview is for iteration — the rebuilt PPTX is the true deliverable. OfficeEditor targets sub-500ms full-deck previews, enabling real-time editing feedback at production scale.



Concept	Detail
➡ Preview Target	<500ms
➡ Fidelity Target	~12% RMSE
➡ Legacy Comparison	15s / 2% RMSE
➡ Deliverable	Rebuilt .pptx
➡ Core Technology	In-process C#

Performance Metrics: Speed Advantage

Benchmark	Speedup
→ AetherLink (15 slides, 1440×810pt)	8.8×
→ pres-pro (16 slides, 960×540pt)	15.8×
→ Per-Slide (AetherLink)	8.8×
→ Per-Slide (pres-pro)	15.9×
→ Cached Single-Slide Preview	~1.5ms



Up to 15.8× Faster Than LibreOffice

OfficeEditor delivers dramatic rendering speed improvements across all tested presentations. For AetherLink (15 slides), OfficeEditor completed in 535.0ms versus LibreOffice at 4690.9ms — an 8.8× speedup. For pres-pro (16 slides), OfficeEditor achieved 111.1ms versus LibreOffice at 1750.3ms — a 15.8× speedup.

Per-slide preview times: OfficeEditor 35.7ms / 6.9ms vs LibreOffice 312.7ms / 109.4ms respectively. Cached single-slide preview latency reaches ~1.5ms, with cold first-request at ~230ms. This validates the core thesis: instant preview enables rapid iteration without waiting for legacy rendering pipelines.

Visual Fidelity Metrics (RMSE)



Metric	Avg RMSE	Page 1 RMSE	Status
 AetherLink (15 slides)	13.20%	5.09%	✓ OK
 pres-pro (16 slides)	17.19%	13.23%	✓ OK
 Target Threshold	<15%	<15%	✓ Met
 Cause: Line Spacing	Minor	Low	Acceptable
 Cause: Font Metrics / Table Stroke	Moderate	Moderate	Acceptable



OfficeEditor

Phase 4 Pitch Deck

Phase 4 Deliverables



Rendering Backend

12 Workstreams · 4 Merge Waves · 844 Tests

- Per-slide preview API
- Font cache system
- Typst emission engine
- Persistent sessions
- Eviction policy
- Placeholder rendering support



Editing Surface

- Element anatomy API
- Instructions API
- Instruction engine
- Style-preserving replacements
- Image fit modes
- TextFitService



Infrastructure

JSON-RPC host & CLI
tools

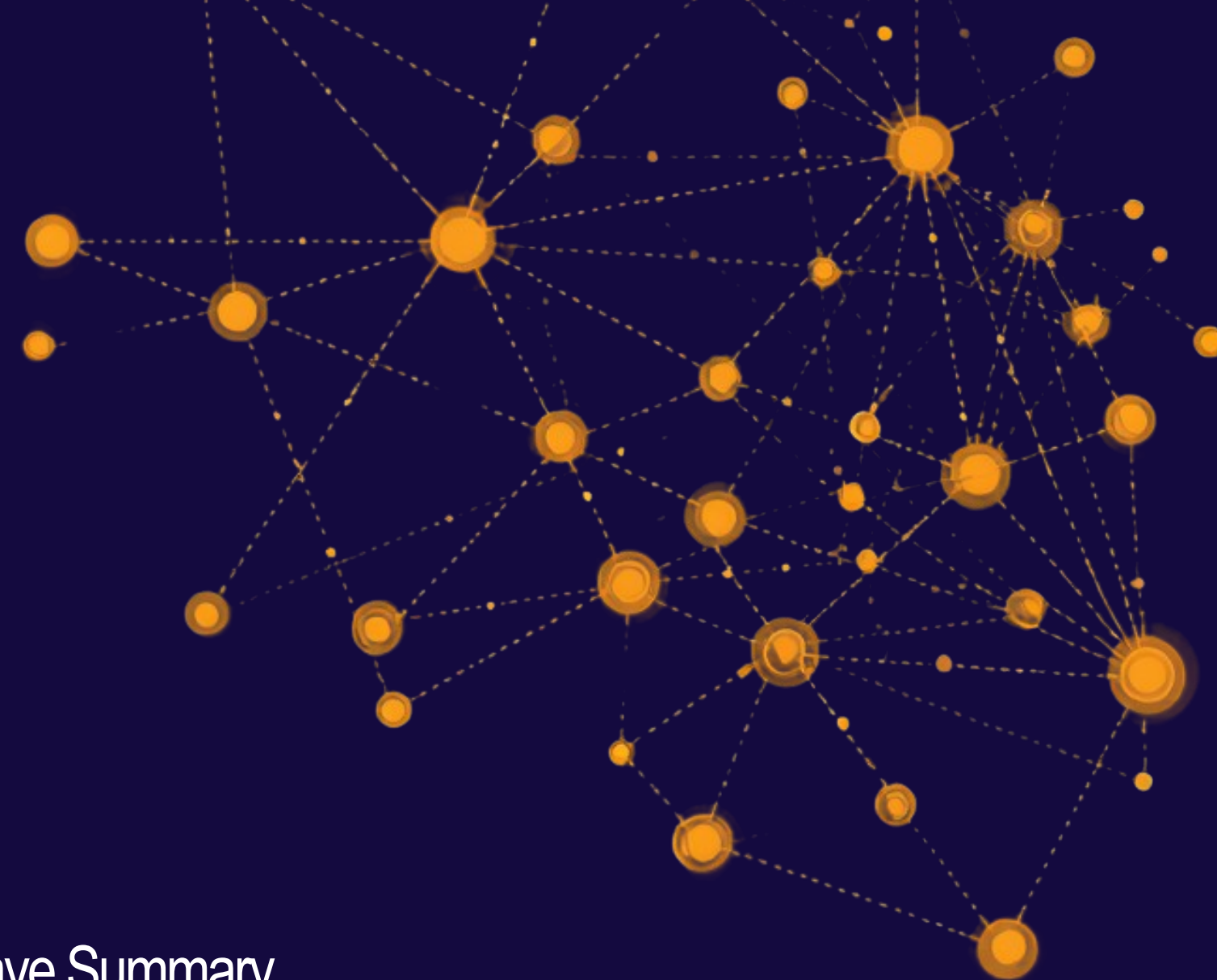
Benchmark harness &
test assemblies



OfficeEditor Phase 4

Internal Presentation

Development Process & Quality Assurance



Workflow Principles

- 01 12 workstreams across 4 merge waves
- 02 One git worktree + feature branch per workstream
- 03 Strict file ownership, zero revert cycles achieved
- 04 844 tests green per merge; 1 regression caught & fixed

Development Wave Summary

01

Waves 1–2:
Rendering backend &
font cache

02

Waves 3–4: Editing
surface & instructions
API

03

Final merge: 844
tests green, zero
regressions
pending



OfficeEditor

Phase 4 Pitch Deck

Architecture & API Endpoints



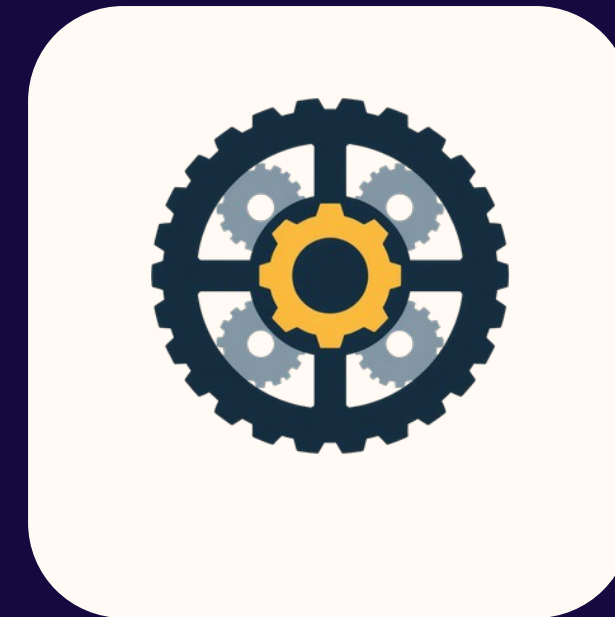
API Interface Design

OfficeEditor exposes a clean REST API layered over a stateless rendering core. The stateful API layer manages deck sessions with a 30-minute TTL, enabling fast cached previews (~1.5ms per slide) after the initial cold load (~230ms). The stateless core handles all rendering logic in-process via C#, with no external process dependencies. Each deck is identified by a `deckId`, and all operations — from anatomy inspection to instruction application and preview generation — are scoped to that identifier. The rebuilt `.pptx` file is always available as the authoritative deliverable output.

Core Endpoints

- 01 POST `/api/decks` → `deckId` (create deck)
- 02 GET `/api/decks/{id}/anatomy` → element map
- 03 POST `/api/decks/{id}/instructions` → apply edits
- 04 GET `/api/decks/{id}/slides/{n}/preview` → PNG/SVG

Integration Highlights & Benefits

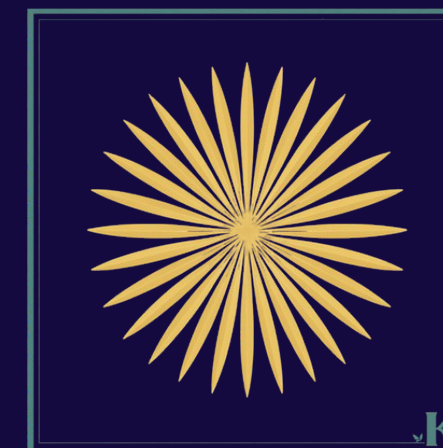


Legacy vs. New Approach

OfficeEditor eliminates the LibreOffice dependency entirely — no licensing costs, no process lifecycle management, and no inter-process communication overhead. The architecture introduces a Stateless Core paired with a stateful API layer, enabling clean session management with a 30-minute TTL. The production stack is lean and modern: .NET 9, DocumentFormat.OpenXml, and a native TypstBridge binary (~37MB). Only one new external dependency is introduced: comemo, a lightweight memoization library. This minimal footprint ensures the system remains easy to deploy, audit, and maintain across environments.

Key Benefits

- 01 No LibreOffice dependency or licensing costs
- 02 No process management overhead
- 03 Stateless Core + Stateful API layer (30-min TTL)
- 04 Production: .NET 9, OpenXml, TypstBridge (~37MB), comemo





OfficeEditor Phase 4

Internal Presentation

Open Issues & Known Challenges



Current Gaps

- 01 RMSE scores are machine-dependent; Linux/x64 testing still pending
- 02 F9 brand.json endpoint wiring incomplete (~20 lines missing)
- 03 Aptos font gap: no metric-compatible open-source clone currently available

Details & Context

- 01 RMSE variance across environments; baseline not yet established on target platform
- 02 No LibreOffice rendering comparison available on Linux target platform yet
- 03 Aptos font substitution causes line-break and advance-width divergence in previews



OfficeEditor

Phase 4 Pitch Deck

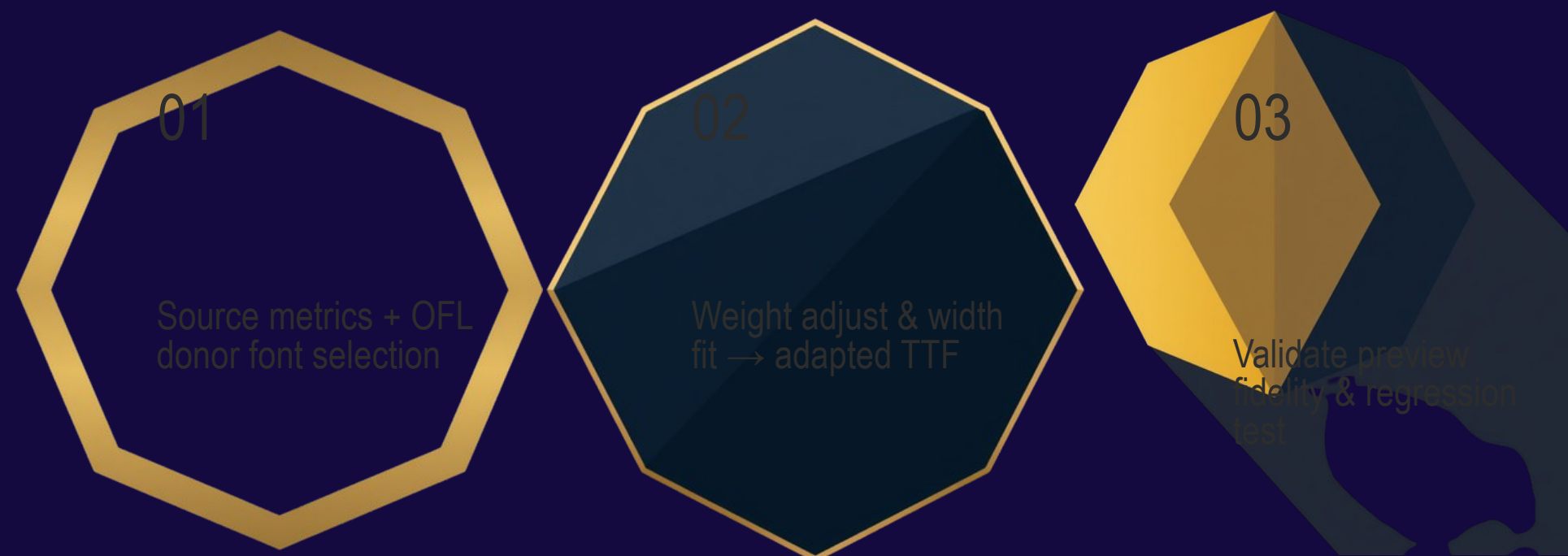
Next Phase: Font Metric-Fitting Roadmap



Font Census & Scope

- 01 ~95% of client decks use Aptos, Calibri, Segoe UI, Arial, Georgia
- 02 Preview-only usage — not embedded in output PPTX
- 03 Regression-tested for advance widths and line-break differences
- 04 Zero metric-compatible open clone for Aptos currently available

Adapted Clone Build Process





OfficeEditor Phase 4

Internal Pitch Deck

Cost Summary

Resource	Count / Detail
➔ Development Workstreams	12
➔ Test Assemblies	5
➔ Total Tests (Green)	844
➔ New Dependencies	1 (comemo)
➔ External Services	Zero



Production Stack: .NET 9 +
DocumentFormat.OpenXml

Phase 4 was delivered with a lean, self-contained resource footprint. All 12 workstreams were completed across 4 merge waves, producing 844 green tests across 5 dedicated test assemblies — with zero regressions left open.

The production runtime requires no LibreOffice installation and introduces only a single new managed dependency: comemo (a memoization utility). The native TypstBridge (~37 MB) is bundled and requires no external service or license.

Core stack: .NET 9, DocumentFormat.OpenXml, and the included TypstBridge native library. No process management overhead, no external rendering services, and no licensing constraints from third-party office suites.



OfficeEditor
Phase 4 Pitch Deck

Thank You



Questions & Feedback

Thank you for your attention and time reviewing the OfficeEditor Phase 4 proposal.
We welcome your questions, feedback, and discussion on the rendering pipeline migration, performance benchmarks, and the roadmap ahead.
OfficeEditor Phase 4 represents a significant step forward — replacing the legacy LibreOffice pipeline with a fast, in-process C# solution delivering 12–15× speed improvements while maintaining acceptable fidelity for iterative workflows.
We look forward to your input as we move into the next phase of development.

Get In Touch



Phone:
+123-456-7890



Email:
hello@reallygreatsite.com



Website:
www.reallygreatsite.com